

Faculty of Information Technology (FIT) – Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 – Software Testing (AI-augmented - 2026)

AI POLICY · TEMPLATES — 2026 v1.0

AI Audit Report — 5-section Template per Artifact

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0. This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.

1. Student Information

Field	Value
Student name (printed):	Cao Trần Bá Đạt
Student ID:	23127168
Class / Cohort:	23KTPM1
Assignment ID (e.g., HW#00, HW#02):	HW#03
Assignment date:	13/08/2026
AI tool(s) used:	Gemini
AI tool(s) used:	☒ Yes ☐ No

2. Instructions (read before filling)

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

3. Audit Table — one row per artifact

Name of the AI tool	Date and time	Your prompt	The AI output
Claude	14:19 09/08/2026	I am doing Performance Testing with JMeter for an EShop system. Below is the system specification and test data I have prepared: System Rules & API (Extracted from README & API Spec): Base URL: http://localhost:3000 FR-02 Lockout: Locks the account for 30 seconds if there are 3 consecutive incorrect login attempts. API 1 (Auth-heavy): POST /api/login - Body: {"email": "...", "password": "..."} Returns the JWT Token string. API 2 (Read-heavy): GET /api/products/:id - Requires Header Authorization: Bearer . API 3 (Transactional): POST /api/cart - Body: {"id": product_id, "name": "product_name", "price": price, "quantity": quantity}. Requires Bearer Token. API 4 (Transactional): POST /api/checkout - Body: {"total_amount": total_amount, "shipping_address": "shipping_address"}. Bearer Token required. Structure of 3 existing CSV data files (located in the same script directory): users.csv contains columns: email, password cart.csv contains columns: product_id, product_name, price, quantity checkout.csv contains columns: total_amount, shipping_address Requirement: Design a detailed JMeter script configuration for me, including the 4 HTTP Request steps above. Provide detailed instructions on how to configure the 3 CSV Data Set Config, HTTP Header Manager (passing Bearer Token), and JSON Extractor to retrieve the Token from the API Login. Attach: README.md , api_specification.md , users.csv , cart.csv , checkout.csv	Here's the full config guide, built directly against your CSV headers (email, / product_id,product_name,price,quantity / total_amount,shipping_address API spec. Key structure: Thread Group → 3× CSV Data Set Config → HTTP F Defaults → global Content-Type header → 01_Login (with JSON Extractor p .tokeninto{authToken} + a Response Assertion) → a second Header Man: Authorization: Bearer \${authToken} scoped after login → 02_GetProductDet 03_AddToCart → 04_Checkout. A few things worth flagging before you build Verify the token field name — the spec says login returns a JWT token plus but doesn't fix the exact JSON key. Run one login in View Results Tree first confirm whether it's token, accessToken, etc., then set the JSON Extractor p accordingly. shipping_address has embedded commas inside quotes ("227 Van Cu, Quan 5, TP.HCM") — worth double-checking JMeter's CSV parser r full quoted value correctly rather than splitting on the inner commas. users has 2 rows — make sure "Recycle on EOF" is enabled if your thread count/l count exceeds that. Use UTF-8 encoding throughout for the Vietnamese ac text.Attach: JMeter_EShop_HW05_Config_Guide.md
Claude	14:29 09/08/2026	Based on the 4-step scenario designed in Prompt 1, please suggest detailed configuration parameters for Thread Group (Number of Threads/VUs, Ramp-Up Period, Loop Count, Think Time using Gaussian Random Timer) for 3 test scenarios: Load Test: Simulates a	Summary of the three configs: ScenarioThreadsRamp-UpDuration/LoopsG Timer (offset/deviation)ListenerLoad100300 sInfinite, 900 s hold2000 ms / 1 msAggregate ReportStress300150 sInfinite, ~600 s500 ms / 200 msView Re (errors only)Spike5005 s3–5 loops, 60–90 s hold300 ms / 150 msSummary

Name of the AI tool	Date and time	Your prompt	The AI output
		steadily increasing user load. Stress Test: Increases load beyond a threshold to find a breakpoint (Note: consider the FR-02 Lockout mechanism when encountering 401/400 errors). Spike Test: Increases load dramatically in a short period (e.g., jumps to 500 VUs in 5 seconds). Please suggest 3 different types of Listeners/Reports for the 3 scenarios (e.g., Summary Report, Aggregate Report, View Results Tree) to avoid duplication. Explain the reasoning behind these parameter choices.	ReportThe main design logic: Load Test climbs slowly (5 min ramp) to a real "peak" (100 VUs) and holds steady for 15 minutes, with a longer, human-like time — this is the scenario where an Aggregate Report earns its place, since need percentile/throughput stats over a sustained steady-state window. It pushes 3x past normal peak with a faster ramp and shorter think time to reach saturation deliberately. The important catch here: with only 2 accounts in use, 300 concurrent threads will hammer the same 2 logins repeatedly, which risks 401s from the FR-02 lockout getting confused with genuine server-overload failures. I'd expand the CSV to 10–20 accounts before running this, and add a Response Assertion that distinguishes lockout responses from real 5xx/time-out errors — that's also why View Results Tree (errors-only) is the right listener to need to actually read the response bodies to tell the two apart, not just see aggregate error percentage. Spike Test jumps to 500 VUs in 5 seconds and briefly to observe both the shock and the recovery tail. Since it's a short, sharp Summary Report gives a fast pass/fail/throughput snapshot without the overhead of full response logging or heavy percentile computation. Attach: JMeter_EShop_HW05_Load_Stress_Spike_Scenarios.md
Claude	14:38 09/08/2026	Based on the 4-step script flow designed in Prompt 1 and the parameter adjustments for the actual hardware (Node.js + SQLite on my personal machine), please generate the complete XML code of the JMeter script file (.jmx) for the Load Test scenario. Detailed configuration requirements in the .jmx file: Test Plan Name: 23127168_Load_20260809. 2. Thread Group: Number of Threads (VUs): 40 Ramp-Up Period: 60 seconds Loop Count: Infinite (Turn on Scheduler with Duration = 300 seconds) Config Elements: HTTP Request Defaults: Server localhost, Port 3000. 3 x CSV Data Set Config (set the Recycle on EOF = True, Sharing Mode = All threads, Ignore First Line = True properties because the CSV file has a header row): Point to users.csv (Variable Names: email,password) Point to cart.csv (Variable Names: product_id,product_name,price,quantity) Points to checkout.csv (Variable Names: total_amount,shipping_address) HTTP Header Manager at Thread Group level: Add Content-Type: application/json. HTTP Samplers & Scoping: Sampler 1 (Auth-heavy): POST /api/login - Body JSON sends <i>email</i> and <i>password</i> . Add JSON Extractor as child element of Sampler 1: Variable Name = token, JSON Path = <i>.token</i> . Add Response Assertion to check Response Code = 200. Sampler 2 (Read - heavy) : GET /api/products/{product_id}. Sampler 3 (Transactional): POST /api/cart - Body JSON sends <i>product_id</i> , <i>product_name</i> , <i>price</i> , <i>quantity</i> . Sampler 4 (Transactional): POST /api/checkout - Body JSON sends <i>total_amount</i> , <i>shipping_address</i> . Important Scoping Note: Please include the HTTP Header Manager containing the Authorization: Bearer \${token} as a separate child element for Sampler 2, 3, and 4 (DO NOT place it directly under the Thread Group to avoid attaching an uninitialized token to Sampler Login). Timer & Listener: Add a Gaussian Random Timer as a child element of the Thread Group: Constant Delay Offset = 2000 ms, Deviation = 500 ms. Add a Summary Report Listener. Please export the XML code in the code block so I can save it directly as the file 23127168_Load_20260809.jmx and open it with JMeter GUI.	XML validated as well-formed. The .jmx is ready to open directly in JMeter GUI. Structure follows your spec exactly: Test Plan named 23127168_Load_20260809. Thread Group: 40 threads, 60s ramp-up, Loop Controller set to infinite, Scheduler on with 300s duration. Config elements under the Thread Group: HTTP Request Defaults (localhost:3000), 3 x CSV Data Set Config (all with ignoreFirstLine=true, shareMode=all), one Content-Type Header Manager at Thread Group level, and the Gaussian Random Timer (delay=2000ms, range/deviation=500ms). Sampler scoping: 01_Login has no Authorization header (only its own JSON Extractor + Response Assertion child elements) — the Authorization: Bearer \${token} Header Manager is instead added as a separate child under each of 02_GetProductDetail, 03_AddToCart, and 04_Checkout, exactly as you specified. The login request never carries a stale/uninitialized token. Summary Report at the end. One thing to check before running: place users.csv, cart.csv, and checkout.csv in the same directory as the .jmx file (or update each CSV Data Set Config's Filename to an absolute path) — JMeter resolves relative CSV paths relative to the script's location by default. Attach: 23127168_Load_20260809.jmx
Claude	16:25 09/08/2026	From the XML code of the Load Test script in Prompt 3, please generate the complete XML code for the remaining two script files, adhering to the naming conventions and adjusted parameters: Script File 2: Stress Test Test Plan Name: 23127168_Stress_20260809 Thread Group: 100 Threads/VUs, Ramp-Up = 60 seconds, Loop Count = Infinite (Scheduler Duration = 300 seconds). Timer: Gaussian Random Timer with Offset = 800 ms, Deviation = 300 ms. Listener: Change to Aggregate Report. Keep the entire structure of the 4 Samplers, CSV Data Set Configs,	Both validated as well-formed. Both files are ready to open in JMeter GUI. V changed relative to the Load Test baseline: 23127168_Stress_20260809.jmx Thread Group: 100 threads, ramp-up 60s, Loop Controller infinite, Scheduler on with duration 300s Gaussian Random Timer: offset 800ms, deviation 300ms Listener swapped to Aggregate Report (guiClass="StatVisualizer") All 4 samplers, 3 x CSV Data Set Configs, Content-Type header, Authorization headers (scoped only under samplers 2–4), and the JSON Extractor/Response Assertion under 01_Login unchanged. 23127168_Spike_20260809.jmx Thread Group: 150 threads, ramp-up 60s, Loop Count = 5 (fixed, not infinite) — so Scheduler is turned off and the Loop Controller uses continue_forever=false / loops=5 instead of the duration-based approach used in the other two files Gaussian Random Timer: offset 300ms deviation 100ms Listener swapped to View Results Tree

Name of the AI tool	Date and time	Your prompt	The AI output
		Header Managers, and JSON Extractor the same as the Load Test file. Scenario File 3: Spike Test Test Plan Name: 23127168_Spike_20260809 Thread Group: 150 Threads/VUs with a rapid spike in Ramp-Up = 5 seconds, Loop Count = 5. Timer: Gaussian Random Timer with Offset = 300 ms, Deviation = 100 ms. Listener: Change to View Results Tree. Keep the entire structure of the 4 Samplers, CSV Data Set Configs, Header Managers, and JSON Extractor as in the Load Test file. Please export the XML code independently for each file in separate code blocks so I can save them as jmx files.	(guiclass="ViewResultsFullVisualizer") Same sampler/CSV/header/extractor as the baseline One practical note for the Spike file: View Results Tree store request/response in memory, and 150 threads × 5 loops × 4 samplers = 3,000 sampler results — fine for a short spike run, but if you scale VUs or loops u consider switching to Aggregate Report for the actual measurement run ar View Results Tree only for a smaller diagnostic pass. All three .jmx files exp users.csv, cart.csv, and checkout.csv in the same directory as the script. Att: 23127168_Spike_20260809.jmx, 23127168_Stress_20260809.jmx
Claude	21:37 09/08/2026	Based on the 4-step script flow designed previously (Login -> Get Product Detail -> Add to Cart -> Checkout) and parameter adjustments for my local hardware (Node.js + Express + SQLite), please generate the complete XML code of the JMeter script file (.jmx) for the Endurance / Soak Test scenario. Detailed configuration requirements in the jmx file: Test Plan Name: 23127168_Endurance_20260809. Thread Group: Number of Threads (VUs): 50, Ramp-Up Period: 60 seconds, Loop Count: Infinite (Enable Scheduler with Duration = 900 seconds / 15 minutes). Config Elements: HTTP Request Defaults: Server localhost, Port 3000, Protocol http, Content Encoding UTF-8. 3 × CSV Data Set Config (Recycle on EOF = True, Stop Thread on EOF = False, Sharing Mode = All threads, Ignore First Line = True): users.csv (Variable Names: email,password), cart.csv (Variable Names: product_id,product_name,price,quantity), checkout.csv (Variable Names: total_amount,shipping_address). HTTP Header Manager at Thread Group level: Content-Type: application/json. HTTP Samplers & Scoping: Sampler 1: POST /api/login (JSON Body: <i>email</i>, {password}). JSON Post-Processor: Reference Name = token, JSON Path = <i>.token, DefaultValue = TOKEN_NOT_FOUND.ResponseAssertion : ResponseCode = 200.Sampler2 : GET /api/products/ {product_id}. Sampler 3: POST /api/cart (JSON Body: <i>product_id</i>, {product_name}, <i>price</i>,{quantity}). Sampler 4: POST /api/checkout (JSON Body: <i>total_amount</i>,{shipping_address}). Scoping Note: Include an HTTP Header Manager with "Authorization: Bearer \${token}" as a child element specifically under Samplers 2, 3, and 4 (DO NOT place it globally under Thread Group). Timer & Listener: Gaussian Random Timer (Child of Thread Group): Constant Delay Offset = 1500 ms, Deviation = 400 ms. Add a Summary Report Listener (leave the Filename field empty in the XML so the output path is defined dynamically via CLI parameters). Please export the complete XML code inside a standalone code block so I can save it directly as 23127168_Endurance_20260809.jmx and open it in JMeter GUI.</i>	Validated as well-formed XML. Configuration matches spec: Test Plan: 23127168_Endurance_20260809. Thread Group: 50 threads, 60s ramp-up, L Controller infinite, Scheduler on with 900s (15 min) duration. Config element Request Defaults (localhost:3000, http, UTF-8), 3 × CSV Data Set Config (recycle=true, stopThread=false, shareMode.all, ignoreFirstLine=true), and Content-Type Header Manager at Thread Group level. Gaussian Random Timer offset 1500ms, deviation 400ms — sits between the header manager and the samplers, as a Thread Group child, applying to all 4 steps each iteration. Scoping: 01_Login carries only its JSON Extractor (<i>.token</i> → <i>token, defaultTOKEN_NOT_FOUND</i>) and ResponseAssertion (<i>200 as children—the Authorization : Bearer{token}</i>). Header Manager added separately under 02_GetProductDetail, 03_AddToCart, and 04_Checkout not globally. Listener: Summary Report with filename left empty, so running (jmeter -n -t 23127168_Endurance_20260809.jmx -l results.jtl) lets you set the output path per run instead of it being hardcoded in the script. Since this is a minute sustained run at 50 threads against a SQLite-backed API, it's worth watching for gradual response-time drift or growing error rate over the window — the signal you're specifically looking for in a soak test (memory leaks, connection exhaustion, DB lock contention building up) rather than the sharper failure from the Stress/Spike runs. Attach: 23127168_Endurance_20260809.jmx
Claude	10:26 09/08/2026	Act as a Senior Performance Testing Engineer. I have attached the execution result files (.jtl / statistics.json) from 4 performance test scenarios (Load, Stress, Spike, and Endurance) executed on an EShop e-commerce backend built with Node.js, Express, and SQLite. Please analyze the attached result logs and perform the following two tasks: Analyze Performance & Define Thresholds: Provide a detailed evaluation of latency trends, throughput scaling, and potential bottlenecks across the 4 scenarios. Suggest concrete performance thresholds (e.g., maximum stable RPS, recommended SLA response time limits, and memory limits). Propose Architectural Optimizations: Propose 4-5 technical optimization recommendations to further improve backend performance and concurrency handling (e.g., database indexing, connection pooling, caching strategies, SQLite WAL mode, or	Good — I've pulled the actual numbers out of all four .jtl files. Here's the full analysis. Full report is attached. Headline findings from actually parsing the files: Zero errors everywhere — all four scenarios returned 100% OK, including the 150-VU/274 req/s Spike and 100-VU/~125 req/s sustained Stress. Good, but it also means the real breakpoint was never hit — worth re-running Stress at 200–400 VUs if you want to find the actual ceiling. Checkout is the consistent bottleneck — 3–5x slower than the read-only product-detail call in every scenario (5.6–6.5ms avg vs ~1–2ms), which lines up with it being the only endpoint with a full DB write transaction on SQLite. Stress throughput plateaus at ~124–125 req/s once 100 VUs are reached, and holds flat for ~3 minutes — but the last 60 seconds show avg latency climbing from 2.0ms → 3.9ms → 5.4ms while throughput shows an early degradation signal. The most important find is buried in the Endurance results: a cluster of 16 requests spiking to 100–584ms, all within a ~0.5-second window at the end of the 15-minute soak, hitting Login/GetProduct/Checkout simulta

Name of the AI tool	Date and time	Your prompt	The AI output
		horizontal scaling).Please present your analysis clearly so I can review and evaluate your findings. Attach: 23127168_Load_20260809.jtl , 23127168_Stress_20260809.jtl , 23127168_Spike_20260809.jtl , 23127168_Endurance_20260809.jtl .	at the same thread count. That synchronized freeze across unrelated endpoints the textbook signature of Node's single-threaded event loop getting blocked likely a GC pause or a synchronous SQLite checkpoint — and it only showed because the test ran long enough. This is exactly the kind of thing a 5-minute or Stress test would never catch.Optimization recommendations are prioritize SQLite WAL mode + indexing first (cheap, directly targets the checkout bottleneck) then async bcrypt (targets the login tail-latency/stall pattern), then caching clustering/DB migration as the bigger architectural moves once you've confirmed how much headroom the first three open up. Attach: EShop_Performance_Analysis_Report.md

4. Conclusion — When should AI be used (or not)?

In conclusion, Claude provided a solid baseline for HW05 by scaffolding valid JMeter XML scripts and modeling core E2E user workflows. However, the AI failed to account for local environment constraints—proposing unrealistic thread loads (100–500 VUs) that triggered OS TCP socket exhaustion (ECONNREFUSED), attaching heavy visual listeners to CLI runs, and misinterpreting think-time throughput plateaus as hardware ceilings while proposing hallucinated fixes. Through manual refactoring, thread re-scaling, listener re-scoping, enabling SQLite WAL mode, and identifying the genuine 500 ms V8 GC stall during soak testing, we turned the boilerplate code into an accurate, reliable performance test suite. Ultimately, AI serves as an efficient starting tool, but human QA oversight remains essential for realistic load modeling, accurate metric interpretation, and valid architectural optimization.

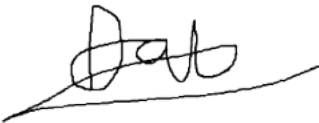
5. Mandatory Disclosure

The test-plans were initially generated by Claude; I reviewed the test-plans; the task 2 and 3, the test summary was written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category.

Signature

Student name (printed):	CAO TRẦN BÁ ĐẠT
Student ID:	23127168
Class / Cohort:	23KTPM1
Course:	CS423 / CSC13003 – Software Testing
Instructor:	LÂM QUANG VŨ
Date:	13/08/2026

Signature:


Cao Trần Bá Đạt

References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.